

23CSE440 - ADVANCED ALGORITHMS AND ANALYSIS

LAB EVALUATION 1

The Dynamic Convex Hull Problem

Graham's Scan Algorithm

Linked Lists, Arrays, AVL Trees, Red-Black Trees

[GitHub](#)

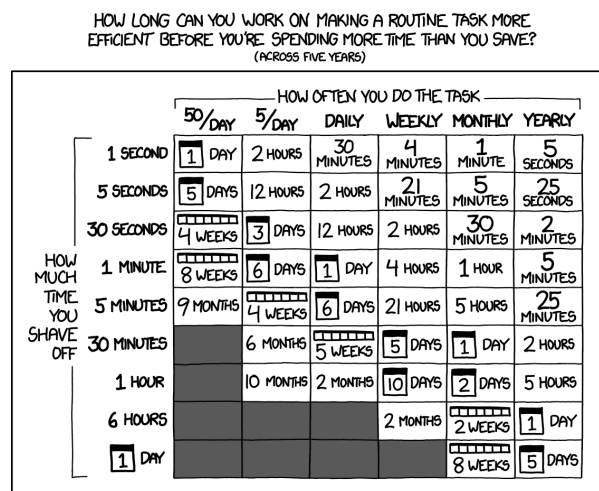


Figure 1: Relevant xkcd

Submitted by: Team 1

Arham Garg	–	CB.SC.U4CSE24206
Bottu Ananya	–	CB.SC.U4CSE24408
Abhipsha Parida	–	CB.SC.U4CSE24601
Akilan S S	–	CB.SC.U4CSE24707

Submitted to:

Dr. Vidhya Balasubramanian

Thursday 16th July, 2026

Contents

1	The Convex Hull Problem	2
1.1	Definition	2
1.2	Existing Approaches	2
1.3	Choice of Algorithm	2
1.4	Dynamic Variant	2
1.5	Algorithm Description	2
2	Behavioural Characteristics	3
3	Formal Proof of Correctness	3
4	Design Choices	6
5	Implementations	6
5.1	Output Stack	6
5.2	Dynamic Sorting	7
5.2.1	Standard Implementation	7
5.2.2	AVL Trees	8
5.2.3	Red-Black Trees	9
6	Randomization and Amortization	10
6.1	Deletion	10
6.2	Insertion	10
7	Benchmarking and Results	11
7.1	Benchmarking Methodology & Environment	11
7.2	Detailed Performance Analysis	11
7.2.1	Build and Hull Construction	11
7.2.2	Normal Dynamic Insertions and Deletions	12
7.2.3	Pivot Insert and Delete: The Rebuilding Phenomenon	12
7.2.4	Memory Allocations & Cache Locality	13
8	Final Realisations	13

1 The Convex Hull Problem

1.1 Definition

In the Convex Hull Problem, we have to find the convex hull of a set of points in computational geometry. The convex hull is the smallest convex set that encloses all the points, forming a convex polygon.

1.2 Existing Approaches

- Divide and Conquer Algorithm – $O(N \log N)$
- Jarvis' Algorithm (Gift Wrapping) – $O(N^2)$
- Graham's Scan Algorithm – $O(N \log N)$
- Monotone Chain Algorithm – $O(N \log N)$
- Quickhull Algorithm – $O(N \log N)$, worst case: $O(N^2)$

1.3 Choice of Algorithm

The algorithm we chose was Graham's Scan, since it had the most optimal time complexity and was also the most interesting to implement in terms of design choices.

1.4 Dynamic Variant

The standard Graham's Scan is static in nature, i.e., the points do not change and only one hull is built. We wanted to tweak the algorithm slightly – in this implementation, the points are dynamic, i.e., points can be inserted or deleted, and the convex hull is reconstructed after each change.

1.5 Algorithm Description

The Graham's Scan algorithm consists of three main steps:

1. **Finding the Pivot Point:** The pivot point is the first vertex of the convex hull and is the bottommost point, i.e., the one with the smallest y -coordinate. In the case of a tie, the point with the smallest x -coordinate is chosen.
2. **Sorting by Polar Angle:** The next step is to calculate the angle between the pivot and each remaining point, and sort the points based on this angle.
3. **Constructing the Hull:** Once the points are sorted, they form a simple closed path. Once we have the closed path, the next step is to traverse the path and remove concave points on this path.

2 Behavioural Characteristics

1. **Orientation and Left Hand Turn** – At every step, the algorithm relies on the cross product of consecutive point vectors to determine turn direction. A positive cross product indicates a counterclockwise (left) turn, a negative value indicates a clockwise (right) turn, and zero indicates collinear points. Only left turns are retained on the hull.
2. **Data Structures** – The static algorithm uses a stack, where points are pushed and popped based on orientation checks. Our dynamic variant instead uses a self-balancing BST (AVL/Red-Black tree) keyed by polar angle, enabling efficient insertion and deletion without a full rebuild.
3. **Cache Locality** – The array/stack-based static implementation benefits from contiguous memory access, giving strong cache locality. The tree-based dynamic implementation trades this off for flexibility, since nodes are scattered in memory, leading to more cache misses during traversal.
4. **Linear Time** – Once the points are sorted by polar angle, the hull construction itself runs in $O(N)$ time, since each point is pushed and popped from the stack at most once. The overall complexity is dominated by the $O(N \log N)$ sort.
5. **Output Stack** – In the static algorithm, the stack directly holds the final hull vertices in order. In the dynamic (tree-based) variant, an in-order traversal of the tree produces the equivalent ordered output.

3 Formal Proof of Correctness

Preliminaries

Let P be a set of points in the Euclidean plane.

Convex Set: A set S is convex if, for any two points $x, y \in S$, the line segment connecting x and y is entirely contained within S .

Convex Hull: The convex hull of a point set P , denoted as $CH(P)$, is defined as the intersection of all convex sets containing P . Equivalently, it is the smallest convex polygon that encloses all points in P . Geometrically, this boundary is analogous to the shape formed by stretching a rubber band around the outermost points of P and letting it snap tight. A point $v \in P$ is a *vertex* of $CH(P)$ if it lies on this strictly convex boundary.

Orientation and ccw Function: For any ordered triplet of points (A, B, C) , the function $ccw(A, B, C)$ determines their orientation. It evaluates to 1 if the sequence forms a strictly counter-clockwise (left) turn, -1 if it forms a clockwise (right) turn, and 0 if the points are collinear.

Initial State Assumption: We assume a base point $v_0 \in P$ has been identified (the point with the lowest y-coordinate, which is guaranteed to be a vertex of $CH(P)$), and that the remaining points $v_1, \dots, v_n$ have been sorted by their polar angle in counter-clockwise order relative to v_0 .

Lemma 1 *The running time of the scan is $O(n)$.*

Lemma 1

Let D_i denote the number of points that are popped on processing p_i . Because each orientation test takes $O(1)$ time, the amount of time spent processing p_i is $O(D_i + 1)$ (the last point is not deleted). Thus, the total time is

$$\sum_{i=1}^n D_i + 1 = n + \sum_{i=1}^n D_i \quad (1)$$

To bound $\sum_i D_i$, each of the point is pushed to stack once, and can be popped only once. Hence

$$\sum_i D_i \leq n \quad (2)$$

Therefore, the total time is $O(2n) = O(n)$ ■

Lemma 2 *Prove that the output stack contains all of the vertices of the convex hull.*

Lemma 2

Every input point gets pushed to the output so it suffices to check that no vertex of $CH(P)$ gets popped from the output. Suppose for contradiction that $v_j \in CH(P)$ and v_j was popped from the output stack. This happens only if there are points v_i, v_k such that $i < j < k$ and

$$\text{ccw}(v_i, v_j, v_k) = -1,$$

or equivalently if

$$\text{ccw}(v_i, v_k, v_j) = 1.$$

By the initial sorting,

$$\text{ccw}(v_0, v_i, v_j) = 1 \quad \text{and} \quad \text{ccw}(v_k, v_0, v_j) = 1.$$

Since

$$x \in \triangle ABC \iff \text{ccw}(A, B, x) = \text{ccw}(B, C, x) = \text{ccw}(C, A, x),$$

we have that

$$v_j \in \triangle v_0 v_i v_k$$

and therefore, $v_j \notin CH(P)$.

Lemma 3 *Prove that the output stack contains only the vertices of the convex hull.*

Lemma 3

We shall prove the above by induction.

Base Case:

$CH(v_0, v_1, v_2)$ is valid, because the sorting guarantees that $ccw(v_0, v_1, v_2) > 0$ and any non-collinear triangle is always convex.

Inductive Hypothesis:

Assume that after k points have been added to the stack, the stack now contains $CH(v_0, \dots, v_k)$. Since it is convex this implies that every consecutive triplet will be convex.

Inductive Step (By Contradiction):

Suppose we push a vertex v_{k+1} into the stack, and this does not result in a convex hull. This can happen only when:

1. The edges of the perimeter intersects
2. The internal angles exceed 180 degrees (i.e., $ccw() < 0$)

But, their edges can never intersect because the points are sorted based on their polar angles. This means that the internal angle is greater than 180 degrees, and forms a right hand turn. But, since the algorithm necessarily pops the item which results in a right hand turn, it is impossible for the point v_k to remain in the stack after it is processed. Hence, we arrive at a contradiction and therefore, only the points on the convex hull remain in the stack.

Lemma 4 *Prove that the vertices are in cyclic order.*

Lemma 4

To be in cyclic order means that there is a point $x \in CH(P)$ such that for every oriented edge $\overline{ab}$ of the hull, we have

$$ccw(a, b, x) \geq 0.$$

We may set $x = v_0$ and observe that this holds for any edge $\overline{v_i v_j}$ with $i < j$ under the initial ordering. Because the ordering is maintained throughout the algorithm, it holds at the end as well.

4 Design Choices

After deciding to make Graham's Scan dynamic, we were clear on one requirement: whichever data structure we chose had to support frequent changes, with easy access, insertion, and deletion. The candidates we considered were linked lists, arrays, and binary trees.

We were also clear that we did not want to sort all the points again on every update. This immediately ruled out arrays, since the simplest and cheapest way to work with an array is to merge in the new elements and sort again, which is expensive. The remaining structures all posed the same underlying question: how do we avoid paying $O(N)$ to search for and insert an element at its correct position, without having to sort again?

This led us to binary search trees (BSTs), which offer $O(\log N)$ insertion, search, and deletion. However, this raised a new concern – what if the BST becomes skewed, degrading performance back to $O(N)$? To guard against this, we decided to use a *self-balancing* BST, narrowing our choices down to AVL trees and Red-Black trees.

Both AVL and Red-Black trees offer $O(\log N)$ insertion, search, and deletion. AVL trees provide faster lookups due to their stricter height balance, while Red-Black trees have the advantage of requiring at most two rotations to restore balance after an insertion (at most three for deletion), compared to the potentially larger number of rotations required by AVL trees.

5 Implementations

When implementing this algorithm, there were primarily two places where a decision is required.

1. The design of the output stack.
2. The design of the dynamic insertion in sorted order.

Since we are exploring the dynamic variant of the algorithm, we gave more emphasis to the sorting. However, we shall still discuss about how the design of the output stack matters.

5.1 Output Stack

Naturally the output stack can be implemented either through an array or a linked list.

1. **Linked List:** Since our algorithm is dynamic, the size of the stack is not known in advance, unlike the offline algorithm where the stack size never exceeds N (the number of points). As new points are inserted at runtime, the stack must grow and shrink accordingly. Linked lists offer $O(1)$ insertion, including insertion at any arbitrary position. However, since the boundary points of the convex hull are traversed frequently when rendering it, this becomes a drawback – linked lists do not offer contiguous memory, so a significant amount of time is spent de-referencing pointers and suffering cache misses.
2. **Array:** The same reasoning applies here, except insertion at the end is amortized $O(1)$, while insertion at an arbitrary position is $O(N)$. However, since we never insert at an arbitrary position – the stack is built incrementally and greedily – we get the best of both worlds: $O(1)$ insertion and contiguous memory access.

5.2 Dynamic Sorting

5.2.1 Standard Implementation

For the standard implementation, we had two candidate data structures: a linked list or an array. Each comes with its own trade-offs in terms of insertion cost, memory overhead, and access pattern.

Linked List: The primary advantage of a linked list is its dynamic size – unlike an array, it does not need to be resized, so we avoid the overhead of allocating a larger block and copying over existing elements. Insertion at a known position costs $O(1)$; however, since the list is unsorted, locating the correct position to insert into requires a linear scan, costing $O(N)$. This makes the total cost of a single insertion $O(N)$, and $O(N^2)$ over N insertions.

Array: When a pivot change triggers a rebuild, all N points are re-inserted one by one into a `std::vector`. The i -th insertion costs $O(\log i)$ for binary search to find the correct position, plus $O(i)$ for shifting all existing elements in contiguous memory to make space. Since `std::vector` handles reallocation internally with a doubling strategy, reallocation is amortized $O(1)$ per insertion – avoiding the explicit $O(N)$ copy overhead that would arise from manually allocating a new array of size $N + k$ and copying all existing elements each time k new points are inserted, which would cost $O(Nk)$ for k insertions and $O(N^2)$ for a full rebuild, albeit with a larger constant than the `std::vector` approach. Summing over all insertions, the total cost is:

$$\sum_{i=1}^N (\log i + i) = \Theta(N \log N) + \Theta(N^2)$$

since

$$\sum_{i=1}^N i = \frac{N(N+1)}{2}$$

by the arithmetic series formula. Since N^2 asymptotically dominates $N \log N$, the overall rebuild complexity is $O(N^2)$, though `std::vector`'s amortized doubling avoids repeated malloc/free overhead, making it significantly faster in practice.

5.2.2 AVL Trees

An AVL tree is a self-balancing binary search tree with $O(\log N)$ height at all times. We use it keyed by the polar angle so that the hull points stay sorted and support fast insertion, deletion, and neighbour queries.

Time Complexity:

- Insertion/Deletion/Search/Predecessor-Successor: $O(\log N)$
- In-order traversal (full hull output): $O(N)$

Dynamic Insertion and Deletion: In insertion, a point is added to the AVL tree by its polar angle in $O(\log N)$ time. Its neighbours are found via predecessor/successor queries, and orientation is checked – if the point lies outside the current hull, neighbouring points are popped in the same manner as static Graham's Scan, but restricted locally to the insertion point. In deletion, the point is removed from the tree in $O(\log N)$ time, and its former neighbours are re-checked for valid orientation, repairing the hull locally if needed.

Pivot Deletion: Deleting the pivot invalidates all polar angles, so a new pivot is recomputed and the tree is rebuilt from scratch in $O(N \log N)$ time.

Pros:

- Local repair on insert/delete instead of full rebuild.
- $O(\log N)$ neighbour lookups via predecessor/successor.

Cons:

- Pivot changes and pivot deletion still require a full $O(N \log N)$ rebuild.
- The pointer-based node structure gives worse cache locality and has higher implementation complexity than a contiguous array.

5.2.3 Red-Black Trees

A red-black tree is a self-balancing binary search tree satisfying the black-root and red-parent properties, with equal black-height along every root-to-leaf path, guaranteeing $O(\log N)$ height at all times. In this implementation, the tree is sorted in Graham's-scan polar order using a three-tier comparator: (1) upper/lower half-plane test, (2) cross product sign, and (3) squared distance from the pivot – computed using `__int128` arithmetic to avoid floating-point rounding errors.

Pivot Management: The first inserted point becomes the pivot, and all subsequent points are stored relative to it as (dx, dy) offsets. If a newly inserted point would become the new pivot (i.e., its (y, x) is smaller than the current pivot's), a lazy rebuild is triggered: all N points are re-inserted relative to the new pivot in $O(N \log N)$ time.

Time Complexity:

- Insertion: $O(\log N)$ normal, $O(N \log N)$ if the pivot changes
- Deletion: $O(\log N)$ normal, $O(N \log N)$ if the pivot is deleted
- Hull construction (in-order traversal + scan): $O(N)$

Pros:

- Fewer rotations per insertion/deletion on average than an AVL tree, since red-black trees allow looser balance.
- Exact integer arithmetic (`__int128`) eliminates floating-point precision errors in orientation tests.

Cons:

- Pivot changes and pivot deletion still require a full $O(N \log N)$ rebuild.
- The pointer-based node structure gives worse cache locality and has higher implementation complexity than a contiguous array.

6 Randomization and Amortization

The benchmarks below sample points from a uniform distribution over $\mathbb{R}^2$ using a random number generator. As discussed, pivot-related insertions and deletions incur a higher cost, so an amortized analysis is presented below.

6.1 Deletion

In search trees, deletion costs $O(\log N)$. However, deleting the pivot requires determining a new pivot and recomputing the search tree in $O(N \log N)$. We now analyse how the frequency of pivot deletions affects the overall time complexity of insertion.

In a uniform sampling, the probability of picking the pivot is:

$$P(v_p) = \frac{1}{n} \quad (3)$$

Now the expected time complexity can be found as:

$$\begin{aligned} \mathbb{E}[\text{deletion}] &= P(\text{worst}) \cdot C(\text{worst}) + P(\text{best}) \cdot C(\text{best}) \\ \mathbb{E}[\text{deletion}] &= \frac{O(n \log n)}{n} + \frac{(n-1) \cdot O(\log n)}{n} \\ \mathbb{E}[\text{deletion}] &= O(\log n) \end{aligned}$$

Therefore, the amortized cost for deletion is still $O(\log n)$.

6.2 Insertion

For insertion, points can be drawn from anywhere in $\mathbb{R}^2$, unlike deletion where points are sampled from the existing n -point set. If the inserted point falls below the current pivot (i.e., has a smaller y -coordinate), a new pivot must be chosen and the search tree rebuilt.

Consider an $m \times m$ region from which points are uniformly sampled and the convex hull is computed. Let the pivot's y -coordinate be k units from the bottom of the region. The probability that a newly inserted point falls below the pivot is:

$$P = \frac{k}{n}$$

Now the expected time complexity can be found as:

$$\begin{aligned} \mathbb{E}[\text{insertion}] &= P(\text{worst}) \cdot C(\text{worst}) + P(\text{best}) \cdot C(\text{best}) \\ \mathbb{E}[\text{insertion}] &= \frac{k * O(n \log n)}{n} + \frac{(n-k) * O(\log n)}{n} \\ \mathbb{E}[\text{insertion}] &= O(k \log n) \end{aligned}$$

This shows that as the pivot moves closer to the bottom of the region, the probability of triggering a search tree recomputation decreases. While sampling from $\mathbb{R}^2$ is theoretically over an infinite domain, in practice computers operate with finite bit-precision, bounding m to a finite value. As k decreases, recomputations become rarer, balancing out the rebuild cost.

7 Benchmarking and Results

7.1 Benchmarking Methodology & Environment

To evaluate the performance of the three dynamic convex hull implementations (Array/Vector, AVL Tree, and Red-Black Tree), we ran a standardised test suite. The benchmark simulates dynamic point workloads with the following environment details:

- **Dataset Size:** 50,000 unique points uniformly distributed in a 2D integer space.
- **Warm-up Runs:** 3 (pre-benchmark warming to eliminate CPU cache and page fault biases).
- **Measured Samples:** 101 runs per benchmark.
- **Mutation Batch Size:** 256 operations per sample (for insert/delete workloads).
- **Compiler:** Clang 21.0.0 (with optimization flags `-O3 -march=native -flto enabled`).
- **Hardware/Platform:** macOS arm64.

The workloads are categorised into:

1. **Build + Hull:** Building the structure from scratch with N points and computing the initial convex hull.
2. **Hull Query:** Fetching the ordered points and executing the Graham's Scan boundary reconstruction.
3. **Normal Insert/Delete:** Inserting or deleting 256 random points (higher/right of pivot) to observe average-case dynamic update performance.
4. **Pivot Insert/Delete:** Inserting or deleting a point that changes the pivot, invalidating all polar angles relative to it and forcing a full structural rebuild.

7.2 Detailed Performance Analysis

7.2.1 Build and Hull Construction

- *Batch Graham Scan* represents the offline baseline where all 50,000 points are sorted statically in a contiguous vector and passed directly to the scan boundary loop. It achieved the fastest overall time with minimal memory allocation overhead, confirming the strength of static arrays for contiguous memory layout and cache optimization.
- *Array build + hull* is the slowest by far. Because the Array implementation inserts points one-by-one, each insertion requires shifting elements in contiguous memory. Summing i shifts over N points leads to a quadratic $\Theta(N^2)$ build time, which dominates the performance.
- *AVL and Red-Black Trees* perform dynamic insertion of N points in $O(N \log N)$ total time. RBT outpaced AVL. This difference arises because RBT has looser height balance requirements, reducing the number of restructuring rotations during sequential dynamic insertions.

Structure	Operation	p50/op	p95/op	mean $\pm$ sd	alloc/op	bytes/op
–	Batch Graham Scan	2.681	2.782	2.690 ms $\pm$ 0.051 ms	8	802032
Array	Build + hull	403.300	405.168	403.437 ms $\pm$ 1.523 ms	40	8614464
Array	Hull query	438.167	449.708	439.702 μ s $\pm$ 10.946 μ s	8	802032
Array	Normal insert	15.482	15.572	15.470 μ s $\pm$ 0.066 μ s	0	0
Array	Normal delete	17.215	17.277	17.206 μ s $\pm$ 0.056 μ s	0	0
Array	Pivot insert	883.042	934.333	889.439 μ s $\pm$ 21.330 μ s	2	2400000
Array	Pivot delete	106.669	106.835	106.692 ms $\pm$ 0.214 ms	1	800000
AVL	Build + hull	8.811	8.945	8.807 ms $\pm$ 0.088 ms	81783	10327104
AVL	Hull query	642.917	662.167	644.308 μ s $\pm$ 11.114 μ s	24	2099168
AVL	Normal insert	131.512	136.395	132.062 ns $\pm$ 3.091 ns	1	80
AVL	Normal delete	112.629	131.836	114.587 ns $\pm$ 8.085 ns	0	0
AVL	Pivot insert	3.109	3.173	3.109 ms $\pm$ 0.046 ms	50017	6097136
AVL	Pivot delete	5.376	5.462	5.373 ms $\pm$ 0.065 ms	50015	6096976
RBT	Build + hull	7.624	7.776	7.638 ms $\pm$ 0.088 ms	81792	10327448
RBT	Hull query	658.667	716.750	666.890 μ s $\pm$ 29.775 μ s	25	2099200
RBT	Normal insert	113.281	117.188	113.613 ns $\pm$ 2.852 ns	1	80
RBT	Normal delete	118.328	130.531	117.286 ns $\pm$ 7.525 ns	0	0
RBT	Pivot insert	2.629	2.698	2.624 ms $\pm$ 0.052 ms	50018	6097168
RBT	Pivot delete	4.618	4.698	4.624 ms $\pm$ 0.086 ms	50016	6097008

Table 1: Per-operation timing (median, 95th percentile, mean $\pm$ standard deviation), allocation counts, and allocated bytes for the array, AVL, and Red–Black tree implementations ($N = 50,000$).

7.2.2 Normal Dynamic Insertions and Deletions

- **Tree Superiority:** Both self-balancing trees achieve nanosecond-level insertion and deletion. The Array, conversely, takes microsecond-level times.
- **100x Speedup:** For a tree, a normal insert/delete runs in $O(\log N)$ which is ≈ 16 pointer transitions. For the Array, locating the index takes $O(\log N)$ binary search, but inserting requires shifting on average $N/2 = 25,000$ elements, leading to $O(N)$ complexity. This creates a 100-fold performance deficit for the Array backend under active point mutations.

7.2.3 Pivot Insert and Delete: The Rebuilding Phenomenon

When a point is added or removed that changes the pivot, the sorted order of polar angles relative to the pivot changes, forcing a complete rebuild of the sorting structure.

- **The Array Rebuild Asymmetry:** Why is Array pivot insert only 889.439 μ s while Array pivot delete takes 106.692 ms?
 - **Pivot Insert:** A new pivot is defined below the old one. We insert it and trigger a rebuild. Because the new pivot’s coordinate is extremely close to the old pivot, the relative polar angles of the existing points remain almost identical. The points are extracted in sorted order and re-inserted. Since they are already sorted, they are appended directly to the end of the array, requiring $O(1)$ shifts. Rebuilding thus takes $O(N)$ linear time.

- **Pivot Delete:** When the old pivot is deleted, a new pivot is chosen from the remaining point set. The polar angles relative to the new pivot shift dramatically, scrambling the sort order. When these points are re-inserted one-by-one into the Array, they are out of order, forcing $O(N)$ shifts for each insertion and resulting in a full quadratic $O(N^2)$ cost.
- **Tree Pivot Rebuilds:** Trees must perform a full tear down and rebuild on pivot changes, taking $O(N \log N)$ time. While AVL and RBT are slower than the Array for the specialised "already-sorted" pivot insertion, they are 20x faster for pivot deletions.

7.2.4 Memory Allocations & Cache Locality

- **Allocation Overhead:** A key drawback of the tree-based approaches is memory overhead. To build a tree of 50,000 points, AVL and RBT perform 81,783 and 81,792 heap allocations, respectively. The Array requires only 40 allocations because it relies on `std::vector`'s amortized doubling capacity, allocating memory in contiguous blocks.
- **Cache Locality Trade-off:** The Array enjoys perfect cache locality (consecutive points sit adjacent in CPU cache lines). Trees distribute nodes across heap segments, causing pointer-chasing and CPU cache misses. This explains why Array hull query (retrieving the sorted points in linear order) is faster than AVL and RBT, even though all three perform the same $O(N)$ Graham's Scan once the points are retrieved.

8 Final Realisations

We expect this lab evaluation to serve as a stepping stone toward understanding how to write performant code, as well as benchmarking and profiling techniques. We also explored the effects of compiler optimizations, auto-vectorization, and related concepts. Additionally, we had planned to cover memory profiling using flame charts during this lab evaluation, but that has been deferred to a future session.

References

- [1] R. L. Graham, *An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set*, Information Processing Letters, 1972.
- [2] Carnegie Mellon University, *15-456 Computational Geometry – Homework 1 Solutions*, <https://www.cs.cmu.edu/afs/cs/academic/class/15456-s10/Homeworks/hw1-answers.pdf>
- [3] University of Maryland, *CMSC 754 – Lecture 2: Convex Hulls I*, <https://www.cs.umd.edu/class/spring2026/cmsc754/Lects/lect02-hulls-1.pdf>
- [4] B. Brewer, G. S. Brodal, H. Wang, *Dynamic Convex Hulls for Simple Paths*, Discrete & Computational Geometry, 75:1151–1186, 2026. <https://cs.au.dk/~gerth/papers/dcg25.pdf>
- [5] Princeton University, *Dynamic Planar Convex Hull*, Course Handout, CS528, <https://www.cs.princeton.edu/courses/archive/fall03/cs528/handouts/dynamic%20planar%20convex%20hull.pdf>
- [6] The Algorithm Archive, *Graham Scan*, https://www.algorithm-archive.org/contents/graham_scan/graham_scan.html
- [7] GeeksforGeeks, *Convex Hull Algorithm*, <https://www.geeksforgeeks.org/dsa/convex-hull-algorithm/>
- [8] Algorithms for Competitive Programming, *Convex Hull Construction*, <https://cp-algorithms.com/geometry/convex-hull.html>